

COMPUTER SCIENCE TRIPOS Part IB – 2023 – Paper 4

7 Cybersecurity (fms27)

The following Python program (imports omitted for brevity) takes a user-supplied `fields.txt` input file containing two text fields, one per line. By concatenating them with template fragments (not fully shown for brevity), it writes out a temporary L^AT_EX file that places those fields in specific positions on the page. It then compiles the L^AT_EX into a pdf, intended to be overprinted onto an existing form. Assume the user has no direct access to the file system: the user interacts with a web page that writes the user-supplied fields, as supplied, to the `fields.txt` file. [Note: In Python, when the end of the file is reached, calling `readline` returns empty string. The default encoding used by `readline` is UTF-8.]

```
1 t_before = "\\documentclass{article} \\begin{document} ..."
2 t_middle = " ... "
3 t_after = " ... \\end{document}"
4 with open("fields.txt", "r") as fields:
5     f0 = fields.readline()
6     f1 = fields.readline()
7 with open("form-content.tex", "w") as latex:
8     latex.write(t_before + f0 + t_middle + f1 + t_after)
9 os.system("pdflatex form-content.tex")
```

- (a) Is it possible to produce a `fields.txt` input file that will cause a malfunction while executing lines 1–8? If yes, produce such input; otherwise justify why this is not possible. State your assumptions explicitly. [3 marks]

Answer: If `fields.txt` contained a malformed UTF-8 sequence, `readline` would raise a `UnicodeDecodeError` exception.

- (b) Is it possible to produce a `fields.txt` input file that will cause a malfunction while executing line 9? If yes, produce such input; otherwise justify why this is not possible. State your assumptions explicitly. [3 marks]

Answer: Any input that, when concatenated with the supplied template fragments, will result in syntactically incorrect L^AT_EX, will cause a crash—from unbalanced curly braces to stray dollar signs, undefined control sequences and plenty more. For example:

```
first field $
second field
```

- (c) Amend the program so that it won't malfunction on the specific inputs you provided in parts (a) or (b). [Note: Ad-hoc fixes that won't fix the general problem are deemed sufficient for full marks here—but see part (e).] [Hint: At least one of parts (a) or (b) admits a “yes” answer.] [3 marks]

Answer:

Part (a): Taking the general view that any input that crashes the program is invalid (a

somewhat selfish and arrogant view that puts the blame entirely on the input and assumes the fault is not with our program... but still), we might choose to reject any such input before feeding it to the L^AT_EX compiler, instead of attempting to sanitise it. We could do that by wrapping the `readline` statements in a `try` block. If we concentrated solely on fixing the Unicode error from the solution note of part (a), our `try` block would catch just the `UnicodeDecodeError` exception. A broader-spectrum countermeasure would catch any exception that occurred during `readline`. An extreme version of this approach would wrap lines 1–8 in a generic `try` block and exit (before launching the L^AT_EX compiler) if *any* exception occurred.

Part (b): Regarding the dollar-sign error we mentioned in the solution note of part (b), we might sanitise the dollar signs by prefixing them with backslashes, for example by inserting the following between lines 6 and 7.

```
f0 = f0.replace("$", r"\$")
f1 = f1.replace("$", r"\$")
```

This fix is narrow-minded and not very robust (all other L^AT_EX syntax errors would be left uncovered) but sufficient in the context of this question. Alternatively we might use the generic approach above of wrapping line 9 in a `try` block as well, but note that this would not guard against the security vulnerability discussed in part (d).

- (d) Is it possible for an attacker to exploit this program to execute an arbitrary command, and under which circumstances? If yes, produce an input file that will execute `/tmp/payload`. If not, justify why this is not possible. State your assumptions explicitly. [3 marks]

Answer: An expert L^AT_EX user will be aware of the obscure `\write18` command (see <https://texfaq.org/FAQ-spawnprog>), use of which is demonstrated by the following malicious input:

```
first field contains exploit: \write18{/tmp/payload}
second field
```

This input will cause `pdflatex` to execute the payload, *provided that* the `\write18` shell spawning facility has not been disabled, as is commonly done in mainstream L^AT_EX distributions precisely to prevent this kind of security problem. For exploit demonstration purposes, the `\write18` command may be (re-)enabled on an ad-hoc basis by running `pdflatex --shell-escape` instead of `pdflatex`.

Here and elsewhere, totally different exploits may also earn full marks if they actually work.

- (e) After the above program caused embarrassment, you are called in to offer security training to the company’s programmers. State the two most important points you are going to make. For each, clearly explain the remedial approaches you recommend, discussing them in the context of the above listing. [8 marks]

Answer:

(The question is pretty open and there is scope to earn the 4+4 marks in many ways, depending on how well the candidate chooses the points and discusses them. For example:)

Point 1: “**Any user input must be treated as potentially malicious and must not be used without prior sanitisation.**” Sanitisation may be based on whitelisting (saying what’s allowed; disallowing anything else) or blacklisting (disabling certain classes of bad

inputs; allowing anything else). The blacklisting approach (like escaping the dollar signs, as we did in the answer to part (b)) is likely to fail to detect some attacks that the defender didn't think about. The whitelisting approach (like allowing only letters and digits) is safer but is likely to exclude valid inputs and thus annoy honest users. Arguably the most principled approach is to correctly quote all user input, but this is never easy. The program does not do that when reading potentially malformed UTF-8 sequences, nor does it do that before inserting the fields into the $\text{\LaTeX}$ template. Sometimes rejecting invalid inputs after the fact (provided the damage hasn't already been done) is simpler than attempting to make them safe.

Point 2: **“Ensure that data (particularly user-supplied data) is never interpreted as code, otherwise this may lead to privilege escalation.”** In part (d) we showed how to persuade $\text{\LaTeX}$ to treat some of its input as code. Following what we said in the previous point, the right thing to do would be to correctly quote user input to ensure that none of it is interpreted as $\text{\LaTeX}$ commands but just as a printable string; however, absent an official library function to do that, written and kept up to date by the implementers of $\text{\LaTeX}$ itself, any homebrew solution is likely to have holes—or at least *develop* holes when $\text{\LaTeX}$ or Python get updated. Note further that the damage caused in part (d) was due not merely to interpreting user input as $\text{\LaTeX}$ commands (already bad) but, much worse, as a particular $\text{\LaTeX}$ command *that spawned a shell* and executed an OS command! This is yet another level of “data as code”. We must actively look for places where such unwanted “data as code” parsing might happen. These are often well hidden, as was the case for `write18`. A frequently exploited one is `os.system`, where a user-supplied argument to a fixed command might contain a semicolon and then another command. But this was not an exploitable vulnerability in this particular case because the string passed to that system call was fixed and had no user-supplied component.
